```
profit = (overhead, costPrice, numberSold, salePrice) ->
  revenue = (salePrice) ->
    (numberSold salePrice) * salePrice
  cost = (salePrice) ->
    overhead + (numberSold salePrice) * costPrice

  (revenue salePrice) – (cost salePrice)
```

The `overhead`, `costPrice`, `numberSold`, and `salePrice` are all values that you either have or can work out using the information you have for a particular product. This will be useful later in this chapter as you continue to make improvements to the original program. First, though, you need to know about *state*.

Gluing existing functions together and building up new functions (and entire programs) are powerful techniques. When you see a programmer staring at the sky proclaiming an epiphany as the result of discovering functional programming, it's usually because of this idea of composing functions (or indeed programs) entirely from other functions. Power requires discipline, particularly where it concerns state. It's time to think about state and why you shouldn't create it where it doesn't already exist.

6.2. State and mutability

Just because CoffeeScript allows you to compose functions doesn't mean that *any* program written in CoffeeScript is written in a functional style. A program can have lots of functions and not be written in a functional style. To quote from Chuck Palahniuk's *Fight Club*, "Sticking feathers up your butt does not make you a chicken."

Functional-style programming in CoffeeScript means composing functions. As you'll see in this section, writing programs by composing functions becomes incredibly difficult if the program manages a lot of explicit state. How does state get into a program, and how can you deal with it? The most important thing to remember is that you should *avoid having variables containing state wherever possible*. What does this mean in practice, though? It's time to find out by looking at variables and side effects, objects, and external state.